

3.6 Hibernate Entity Lifecycle, Entity Manager and Persistence Context

• Hibernate Entity Life Cycle:

• What is an Entity Life Cycle?

- In Hibernate, an **entity** is a Java object mapped to a database table.
- The **entity life cycle** describes the different states an entity goes through **from creation to deletion** while interacting with:
 - Hibernate
 - Persistence Context (Session / EntityManager)
 - Database
- Hibernate defines **four main states** of an entity.

• Persistence Context:

- A Persistence Context is:
 - A **container** managed by Hibernate
 - It **tracks** all managed (persistent) **entities**
 - Ensures **identity**, **dirty checking**, and **automatic synchronization**.
- In Hibernate:
 - **Session** (native Hibernate)
 - **EntityManager** (JPA)
- Both represent a **persistence context**.

• Entity States in Hibernate:

1) Transient State:

- An entity is in **Transient state** when:
 - It is created using **new**.
 - It is **not associated** with any *Hibernate session*.
 - It does **not exist** in the database
- **Example:**

```
Employee emp = new Employee();  
emp.setName("John");
```
- **Characteristics:**
 - *Not tracked by Hibernate*
 - *No database row exists*
 - Changes are *ignored by Hibernate*
 - *No persistence context* involvement

2) Persistent State:

- An entity is in **Persistent state** when:
 - It is associated with an **active session**.
 - **Hibernate tracks** it inside the **persistence context**.
 - It corresponds to a **database row**.

3.6 Hibernate Entity Lifecycle, Entity Manager and Persistence Context

- **Example:**

```
Session session = sessionFactory.openSession();
session.beginTransaction();
```

```
Employee emp = session.get(Employee.class, 1L);
```

```
/* or */
```

```
Employee emp = new Employee();
session.save(emp);
```

```
/* or */
```

use *@Transactional* annotation

- **Characteristics:**

- Managed by **Hibernate**
- Any change is **automatically detected** (dirty checking)
- No explicit **update()** needed
- **Synchronized** with DB on **flush()** or **commit()**

3) Detached State:

- An entity is in **Detached state** when:
 - It was **persistent earlier**.
 - The session is **closed** or **cleared**.
 - It is **no longer** managed by Hibernate.

- **Example:**

```
session.close();
emp.setName("Updated Name");
```

- **Characteristics:**

- Exists in database.
- Hibernate does **not track** changes.
- Changes are **NOT auto-saved**.
- Can be **reattached** using **merge()** or **update()**.

4) Removed (Deleted) State:

- An entity is in **Removed state** when:
 - It is marked for **deletion**.
 - Hibernate **schedules DELETE** operation.

- **Example:**

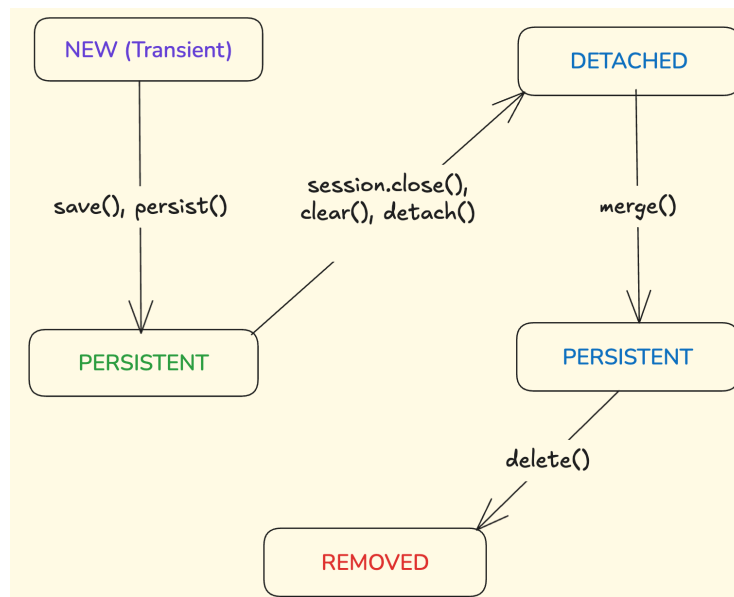
```
session.delete(emp);
```

- **Characteristics:**

- Still **managed** until **transaction commit**.
- **Deleted** from DB on **flush/commit**.
- After **commit** → entity becomes **detached**.

3.6 Hibernate Entity Lifecycle, Entity Manager and Persistence Context

- Entity State Transition Diagram:



- Dirty Checking:

- Hibernate automatically tracks changes in **persistent entities**.
- On transaction **commit**:
 - Hibernate compares **snapshot** with **current state**.
 - Generates **SQL UPDATE** if changes are detected.

- Why Entity Lifecycle Matters:

- Understanding entity lifecycle helps you:
 - **Avoid** unnecessary **DB queries**.
 - Prevent **LazyInitializationException**.
 - Handle **detached objects** properly.
 - **Optimize** performance.
 - Write correct **transactional logic**.